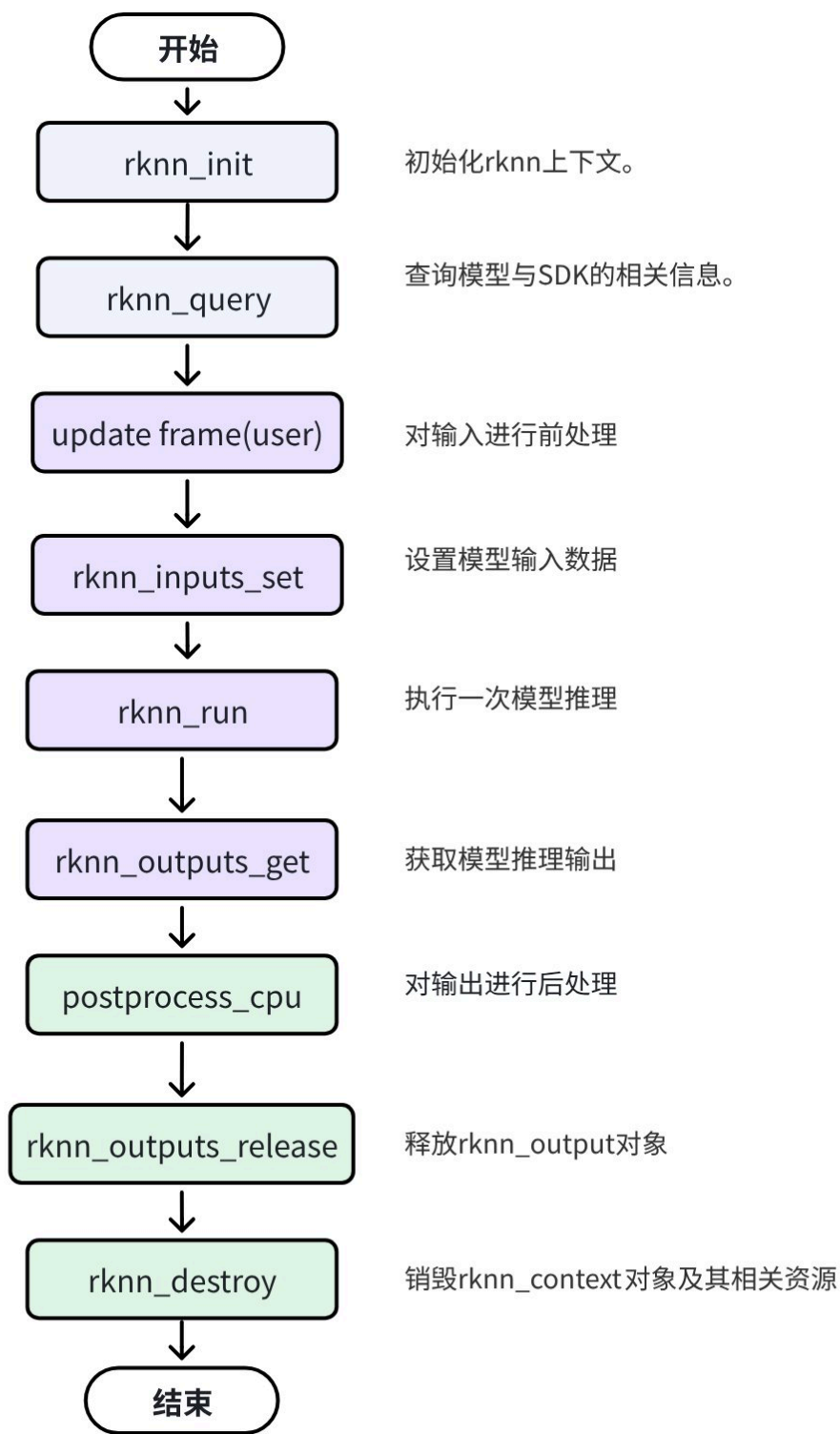


25.RKNPU2—通用C API

1. 通用C API调用流程



表格

步骤	函数名	核心动作	数据流向	备注
1	rknn_init	加载模型	-	仅执行一次
2	rknn_query	获取规格	-	决定预处理参数
3	Pre-process	图像变换	CPU内存内	Resize, Normalize
4	rknn_inputs_set	拷贝输入	CPU → NPU	通用API特征
5	rknn_run	矩阵运算	NPU内部	耗时主要在这里
6	rknn_outputs_get	拷贝输出	NPU → CPU	通用API特征
7	Post-process	解析结果	CPU内存内	Softmax, NMS
8	rknn_outputs_release	释放内存	-	防止内存泄漏

通用API调用流程示例代码：

代码块

```
1 // Init RKNN model
2 ret = rknn_init(&ctx, model, model_len, 0, NULL);
3
4 // Get Model Input Output Number
5 rknn_input_output_num io_num;
6 ret = rknn_query(ctx, RKNN_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));
7
8 // Get Model Input Info
9 rknn_tensor_attr input_attrs[io_num.n_input];
10 memset(input_attrs, 0, sizeof(input_attrs));
11 for (int i = 0; i < io_num.n_input; i++)
12 {
13     input_attrs[i].index = i;
14     ret = rknn_query(ctx, RKNN_QUERY_INPUT_ATTR, &(input_attrs[i]),
15 sizeof(rknn_tensor_attr));
16 }
17
18 // Get Model Output Info
19 rknn_tensor_attr output_attrs[io_num.n_output];
20 memset(output_attrs, 0, sizeof(output_attrs));
21 for (int i = 0; i < io_num.n_output; i++)
22 {
23     output_attrs[i].index = i;
```

```

23     ret = rknn_query(ctx, RKNN_QUERY_OUTPUT_ATTR, &(output_attrs[i]),
    sizeof(rknn_tensor_attr));
24 }
25 rknn_input inputs[io_num.n_input];
26 rknn_output outputs[io_num.n_output];
27 memset(inputs, 0, sizeof(inputs));
28 memset(outputs, 0, sizeof(outputs));
29
30 // Pre-process
31 // Read Image
32 image_buffer_t src_image;
33 memset(&src_image, 0, sizeof(image_buffer_t));
34 ret = read_image(image_path, &src_image);
35
36 // Set Input Data
37 inputs[0].index = 0;
38 inputs[0].type = RKNN_TENSOR_UINT8;
39 inputs[0].fmt = RKNN_TENSOR_NHWC;
40 inputs[0].size = src_image.size;
41 inputs[0].buf = src_image.virt_addr;
42 ret = rknn_inputs_set(rknn_ctx, io_num.n_input, inputs);
43
44 // Run
45 ret = rknn_run(rknn_ctx, nullptr);
46
47 // Get Output Data
48 ret = rknn_outputs_get(rknn_ctx, io_num.n_output, outputs, NULL);
49
50 // Post-process
51 post_process(outputs, results);
52
53 // Release RKNN Output
54 rknn_outputs_release(rknn_ctx, io_num.n_output, outputs);
55 33

```

2. 环境运行搭建

这里参考前面第8节，快速体验rknn_demo那篇，就不详细写了。

1. 设置工具链

代码块

```

1 export GCC_COMPILER=/opt/tool_chain/gcc-arm-10.3-2021.07-x86_64-aarch64-none-
    linux-gnu/bin/aarch64-none-linux-gnu

```

2.构建

代码块

```
1  ./build-linux.sh -t rk3588 -a aarch64 -b Release
```

3.将编译出来的install目录下的文件拷贝到开发板上

代码块

```
1  scp -r rknn_common_test_Linux/ root@192.168.137.142:/userdata
```

4.开发板执行

代码块

```
1  cd rknn_common_test_Linux/  
2  export LD_LIBRARY_PATH=./lib  
3  ./rknn_common_test model/RK3588/mobilenet_v1.rknn model/dog_224x224.jpg
```

3. 代码测试

16_rknpu2_common_test

查看结果

```
(rknn-toolkitlite2) root@topeet:/userdata/rknn_common_test_Linux# ./rknn_common_test model/RK3588/mobilenet_v1.rknn model/dog_224x224.jpg  
rknn_api/rknnrt version: 2.3.2 (429f97ae6b@2025-04-09T09:09:27), driver version: 0.9.8  
model input num: 1, output num: 1  
input tensors:  
  index=0, name=input, n_dims=4, dims=[1, 224, 224, 3], n_elems=150528, size=150528, fmt=NHWC, type=INT8, qnt_type=AFFINE, zp=0, scale=0.007812  
output tensors:  
  index=0, name=MobilenetV1/Predictions/Reshape_1, n_dims=2, dims=[1, 1001, 0, 0], n_elems=1001, size=2002, fmt=UNDEFINED, type=FP16, qnt_type=AFFINE, zp=0, scale=1.000000  
custom string:  
Begin perf ...  
  0: Elapse Time = 3.09ms, FPS = 323.73  
---- Top5 ----  
0.884766 - 156  
0.054016 - 155  
0.003677 - 205  
0.002974 - 284  
0.000189 - 285
```

第一部分：环境准备与工具函数

代码块

```
1  // 获取当前系统时间，返回微秒级时间戳，用于后续计算推理耗时  
2  static inline int64_t getCurrentTimeUs()  
3  {  
4      struct timeval tv;
```

```

5     gettimeofday(&tv, NULL);
6     return tv.tv_sec * 1000000 + tv.tv_usec;
7 }
8
9 // 从模型输出的概率数组中提取 TopN 个最高概率及其对应的类别索引
10 // pfProb: 模型输出的原始概率数组
11 // pfMaxProb: 用于存储找出的前N个最大概率值
12 // pMaxClass: 用于存储前N个最大概率对应的类别ID
13 // outputCount: 输出的总类别数
14 // topNum: 需要提取的Top数量
15 static int rknn_GetTopN(float* pfProb, float* pfMaxProb, uint32_t* pMaxClass,
uint32_t outputCount, uint32_t topNum)
16 {
17     uint32_t i, j;
18     uint32_t top_count = outputCount > topNum ? topNum : outputCount;
19
20     // 初始化最大概率数组为极小值, 类别索引为-1
21     for (i = 0; i < topNum; ++i) {
22         pfMaxProb[i] = -FLT_MAX;
23         pMaxClass[i] = -1;
24     }
25
26     // 双重循环遍历寻找TopN
27     for (j = 0; j < top_count; j++) {
28         for (i = 0; i < outputCount; i++) {
29             // 如果当前类别已经在已找到的TopN中, 则跳过
30             if ((i == *(pMaxClass + 0)) || (i == *(pMaxClass + 1)) || (i == *
(pMaxClass + 2)) || (i == *(pMaxClass + 3)) ||
31                 (i == *(pMaxClass + 4))) {
32                 continue;
33             }
34
35             // 如果当前概率大于已记录的第j个最大概率, 则更新
36             if (pfProb[i] > *(pfMaxProb + j)) {
37                 *(pfMaxProb + j) = pfProb[i];
38                 *(pMaxClass + j) = i;
39             }
40         }
41     }
42
43     return 1;
44 }

```

这份代码是通用 C API 的标准实现，非常适合用于教学演示。为了方便你制作课件，我将代码按照功能逻辑进行了分段，并提供了详细的注释和对应的原理解释。你可以直接将这些内容复制到 PPT 或文档中。

第一部分：环境准备与工具函数

这部分代码包含了必要的头文件引用以及两个辅助函数：一个是用于性能测试的高精度计时器，另一个是用于后处理的 Top-N 算法。

代码块

```
1  #include "opencv2/core/core.hpp"
2  #include "opencv2/imgcodecs.hpp"
3  #include "opencv2/imgproc.hpp"
4  #include "rknn_api.h"
5
6  #include <float.h>
7  #include <stdio.h>
8  #include <stdlib.h>
9  #include <string.h>
10 #include <sys/time.h>
11
12 using namespace std;
13 using namespace cv;
14
15 /*-----
16                      Functions
17 -----*/
18 // 获取当前系统时间，返回微秒级时间戳，用于后续计算推理耗时
19 static inline int64_t getCurrentTimeUs()
20 {
21     struct timeval tv;
22     gettimeofday(&tv, NULL);
23     return tv.tv_sec * 1000000 + tv.tv_usec;
24 }
25
26 // 从模型输出的概率数组中提取 TopN 个最高概率及其对应的类别索引
27 // pfProb: 模型输出的原始概率数组
28 // pfMaxProb: 用于存储找出的前N个最大概率值
29 // pMaxClass: 用于存储前N个最大概率对应的类别ID
30 // outputCount: 输出的总类别数
31 // topNum: 需要提取的Top数量
32 static int rknn_GetTopN(float* pfProb, float* pfMaxProb, uint32_t* pMaxClass,
33     uint32_t outputCount, uint32_t topNum)
34 {
35     uint32_t i, j;
36     uint32_t top_count = outputCount > topNum ? topNum : outputCount;
37
38     // 初始化最大概率数组为极小值，类别索引为-1
39     for (i = 0; i < topNum; ++i) {
```

```

39         pfMaxProb[i] = -FLT_MAX;
40         pMaxClass[i] = -1;
41     }
42
43     // 双重循环遍历寻找TopN
44     for (j = 0; j < top_count; j++) {
45         for (i = 0; i < outputCount; i++) {
46             // 如果当前类别已经在已找到的TopN中, 则跳过
47             if ((i == *(pMaxClass + 0)) || (i == *(pMaxClass + 1)) || (i == *
(pMaxClass + 2)) || (i == *(pMaxClass + 3)) ||
48                 (i == *(pMaxClass + 4))) {
49                 continue;
50             }
51
52             // 如果当前概率大于已记录的第j个最大概率, 则更新
53             if (pfProb[i] > *(pfMaxProb + j)) {
54                 *(pfMaxProb + j) = pfProb[i];
55                 *(pMaxClass + j) = i;
56             }
57         }
58     }
59
60     return 1;
61 }

```

- 高精度计时：`getCurrentTimeUs()` 使用 `gettimeofday` 获取微秒级时间戳。在嵌入式开发中，毫秒级的 `clock()` 往往不够精确，无法准确衡量 NPU 几十毫秒甚至几毫秒的推理耗时。
- Top-N 算法：这是一个经典的“选择排序”变体。模型输出通常是一个包含 1000 个浮点数的数组（对应 ImageNet 的 1000 类）。我们需要找出数值最大的前 5 个数及其下标。这里没有使用 STL 的 `std::sort`，而是手动实现了双重循环，这在资源受限的嵌入式环境中是一种常见的轻量级做法

第二部分：模型初始化与属性查询

代码块

```

1  /*-----
2
3      Main Functions
4  -----*/
5  int main(int argc, char* argv[])
6  {
7      // 检查命令行参数是否合法, 至少需要传入模型路径和输入图片路径
8      if (argc < 3) {
9          printf("Usage:%s model_path input_path [loop_count]\n", argv[0]);
10         return -1;

```

```
10     }
11
12     char* model_path = argv[1]; // 获取模型文件路径
13     char* input_path = argv[2]; // 获取输入图片路径
14
15     // 获取循环测试次数，默认为1次
16     int loop_count = 1;
17     if (argc > 3) {
18         loop_count = atoi(argv[3]);
19     }
20
21     rknn_context ctx = 0; // 定义RKNN上下文句柄，后续所有NPU操作都依赖它
22
23     // 初始化 RKNN 模型，加载模型文件并分配 NPU 资源
24     int ret = rknn_init(&ctx, model_path, 0, 0, NULL);
25     if (ret < 0) {
26         printf("rknn_init fail! ret=%d\n", ret);
27         return -1;
28     }
29
30     // 查询并打印 SDK 和底层驱动器的当前版本信息
31     rknn_sdk_version sdk_ver;
32     ret = rknn_query(ctx, RKNN_QUERY_SDK_VERSION, &sdk_ver, sizeof(sdk_ver));
33     if (ret != RKNN_SUCC) {
34         printf("rknn_query fail! ret=%d\n", ret);
35         return -1;
36     }
37     printf("rknn_api/rknnrt version: %s, driver version: %s\n",
38           sdk_ver.api_version, sdk_ver.drv_version);
39
40     // 查询模型的输入和输出张量数量
41     rknn_input_output_num io_num;
42     ret = rknn_query(ctx, RKNN_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));
43     if (ret != RKNN_SUCC) {
44         printf("rknn_query fail! ret=%d\n", ret);
45         return -1;
46     }
47     printf("model input num: %d, output num: %d\n", io_num.n_input,
48           io_num.n_output);
49
50     // 遍历并打印所有输入张量的属性信息
51     printf("input tensors:\n");
52     rknn_tensor_attr input_attrs[io_num.n_input];
53     memset(input_attrs, 0, io_num.n_input * sizeof(rknn_tensor_attr));
54     for (uint32_t i = 0; i < io_num.n_input; i++) {
55         input_attrs[i].index = i;
```



```

54     ret = rknn_query(ctx, RKNN_QUERY_INPUT_ATTR, &(input_attrs[i]),
sizeof(rknn_tensor_attr));
55     if (ret < 0) {
56         printf("rknn_init error! ret=%d\n", ret);
57         return -1;
58     }
59     dump_tensor_attr(&input_attrs[i]);
60 }
61
62 // 遍历并打印所有输出张量的属性信息
63 printf("output tensors:\n");
64 rknn_tensor_attr output_attrs[io_num.n_output];
65 memset(output_attrs, 0, io_num.n_output * sizeof(rknn_tensor_attr));
66 for (uint32_t i = 0; i < io_num.n_output; i++) {
67     output_attrs[i].index = i;
68     ret = rknn_query(ctx, RKNN_QUERY_OUTPUT_ATTR, &(output_attrs[i]),
sizeof(rknn_tensor_attr));
69     if (ret != RKNN_SUCC) {
70         printf("rknn_query fail! ret=%d\n", ret);
71         return -1;
72     }
73     dump_tensor_attr(&output_attrs[i]);
74 }
75
76 // 查询并打印模型附带的自定义字符串信息
77 rknn_custom_string custom_string;
78 ret = rknn_query(ctx, RKNN_QUERY_CUSTOM_STRING, &custom_string,
sizeof(custom_string));
79 if (ret != RKNN_SUCC) {
80     printf("rknn_query fail! ret=%d\n", ret);
81     return -1;
82 }
83 printf("custom string: %s\n", custom_string.string);

```

- Context 机制： `rknn_init` 是整个流程的核心。它不仅加载了 `.rknn` 模型文件，还会根据模型结构在 NPU 上分配计算图资源。返回的 `ctx` 句柄相当于一个“会话”，后续所有的操作都必须指定这个句柄。
- 动态查询属性：为什么不能硬编码输入输出尺寸？因为同一个 API 可能要跑不同的模型（如 YOLOv5 和 MobileNet）。通过 `rknn_query`，我们可以动态获知模型要求的输入形状（Dims）、数据类型（Type）和内存布局（Format），从而编写通用的预处理代码。
- 调试信息：`dump_tensor_attr` 打印的信息非常关键，特别是 `fmt`（NHWC/NCHW）和 `qnt_type`（量化类型），这决定了我们在 CPU 端该如何准备数据。

第三部分：图像预处理与数据封装

代码块

```
1 // 定义输入数据的默认类型和内存布局
2 unsigned char* input_data = NULL;
3 rknn_tensor_type input_type = RKNN_TENSOR_UINT8;
4 rknn_tensor_format input_layout = RKNN_TENSOR_NHWC;
5
6 // 根据模型要求的输入格式，获取目标宽高和通道数
7 int req_height = 0;
8 int req_width = 0;
9 int req_channel = 0;
10
11 switch (input_attrs[0].fmt) {
12 case RKNN_TENSOR_NHWC:
13     req_height = input_attrs[0].dims[1];
14     req_width = input_attrs[0].dims[2];
15     req_channel = input_attrs[0].dims[3];
16     break;
17 case RKNN_TENSOR_NCHW:
18     req_height = input_attrs[0].dims[2];
19     req_width = input_attrs[0].dims[3];
20     req_channel = input_attrs[0].dims[1];
21     break;
22 default:
23     printf("meet unsupported layout\n");
24     return -1;
25 }
26
27 // 使用 OpenCV 读取原始图片
28 int height = 0;
29 int width = 0;
30 int channel = 0;
31
32 cv::Mat orig_img = imread(input_path, cv::IMREAD_COLOR);
33 if (!orig_img.data) {
34     printf("cv::imread %s fail!\n", input_path);
35     return -1;
36 }
37
38 // 将 BGR 格式转换为 RGB 格式 (Caffe模型可能不需要此步骤)
39 cv::Mat orig_img_rgb;
40 cv::cvtColor(orig_img, orig_img_rgb, cv::COLOR_BGR2RGB);
41
42 // 如果图片尺寸与模型要求不符，则进行缩放 (Resize)
43 cv::Mat img = orig_img_rgb.clone();
44 if (orig_img.cols != req_width || orig_img.rows != req_height) {
```

```

45     printf("resize %d %d to %d %d\n", orig_img.cols, orig_img.rows,
req_width, req_height);
46     cv::resize(orig_img_rgb, img, cv::Size(req_width, req_height), 0, 0,
cv::INTER_LINEAR);
47 }
48 input_data = img.data;
49 if (!input_data) {
50     return -1;
51 }
52
53 // 【通用API核心】准备输入数据对象
54 // 在通用API中，我们只需要构建一个描述符，指向CPU端的普通内存即可
55 rknn_input inputs[1];
56 memset(inputs, 0, sizeof(inputs));
57 inputs[0].index = 0;
58 inputs[0].type = input_type;
59 inputs[0].size = req_width * req_height * req_channel;
60 inputs[0].fmt = input_layout;
61 inputs[0].buf = input_data; // 直接指向 OpenCV Mat 的数据指针

```

- 预处理流程：标准的深度学习预处理三步曲：Read -> Convert Color -> Resize。OpenCV 默认读取的是 BGR 格式，而大多数训练好的模型（如 TensorFlow/PyTorch 导出的）期望的是 RGB 格式，所以必须转换。
- 通用 API 的特点：请注意 `inputs[0].buf = input_data` 这一行。在通用 API 中，`buf` 指向的是 CPU 的普通堆内存（Heap Memory）。SDK 并不关心这块内存是怎么来的，也不要求内存对齐。SDK 会在调用 `rknn_inputs_set` 时，自动把这块内存里的数据拷贝到 NPU 的内部 SRAM/DRAM 中。这简化了开发者的负担，但牺牲了一点性能。

第四部分：推理执行与结果获取

代码块

```

1 // 开始性能测试循环
2 printf("Begin perf ...\n");
3 for (int i = 0; i < loop_count; ++i) {
4     int64_t start_us = getCurrentTimeUs(); // 记录推理开始时间
5
6     // 【通用API核心】设置输入数据
7     // SDK 内部会执行：CPU内存 -> NPU内部内存 的拷贝操作
8     ret = rknn_inputs_set(ctx, io_num.n_input, inputs);
9     if (ret < 0) {
10         printf("rknn_inputs_set fail! ret=%d\n", ret);
11         return -1;
12     }
13

```

```

14 // 触发 NPU 执行推理
15 ret = rknn_run(ctx, NULL);
16 if (ret < 0) {
17     printf("rknn run error %d\n", ret);
18     return -1;
19 }
20
21 // 【通用API核心】获取输出结果
22 // SDK 内部会执行: NPU内部内存 -> CPU内存 的拷贝操作
23 // 并且会自动处理反量化 (Dequantize) 和数据类型转换
24 rknn_output outputs[io_num.n_output];
25 memset(outputs, 0, sizeof(outputs));
26 // want_float = 1 表示希望SDK帮我们将量化后的INT8数据转换为FLOAT32
27 for (uint32_t j = 0; j < io_num.n_output; ++j) {
28     outputs[j].want_float = 1;
29 }
30
31 ret = rknn_outputs_get(ctx, io_num.n_output, outputs, NULL);
32 if (ret < 0) {
33     printf("rknn_outputs_get fail! ret=%d\n", ret);
34     return -1;
35 }
36
37 int64_t elapse_us = getCurrentTimeUs() - start_us; // 计算耗时
38
39 // 解析输出结果, 提取 Top5 类别
40 uint32_t topNum = 5;
41 for (uint32_t j = 0; j < io_num.n_output; j++) {
42     uint32_t MaxClass[topNum];
43     float fMaxProb[topNum];
44     // 注意: outputs[j].buf 现在是 SDK 帮我们分配并填充好数据的 CPU 内存
45     float* buffer = (float*)outputs[j].buf;
46     uint32_t sz = output_attrs[j].n_elems;
47     int top_count = sz > topNum ? topNum : sz;
48
49     // 调用 TopN 提取函数
50     rknn_GetTopN(buffer, fMaxProb, MaxClass, sz, topNum);
51
52     // 打印 TopN 结果
53     printf("---- Top%d ----\n", top_count);
54     for (int k = 0; k < top_count; k++) {
55         printf("%8.6f - %d\n", fMaxProb[k], MaxClass[k]);
56     }
57 }
58
59 // 【通用API核心】释放输出内存
60 // 因为 rknn_outputs_get 内部 malloc 了内存, 所以必须手动释放

```

```

61         rknn_outputs_release(ctx, io_num.n_output, outputs);
62
63         // 打印单次推理的耗时 (毫秒) 和帧率 (FPS)
64         printf("%4d: Elapse Time = %.2fms, FPS = %.2f\n", i, elapse_us /
1000.f, 1000.f * 1000.f / elapse_us);
65     }

```

- 输入流 (rknn_inputs_set): 这是通用 API 的标志性函数。它将我们在上一节准备好的 inputs 结构体传给 SDK。SDK 拿到后, 会根据 index 找到对应的 NPU 输入端口, 然后把 buf 里的数据搬运过去。代价: 每次推理都要拷贝一次输入数据。
- 输出流 (rknn_outputs_get): 这是另一个标志性函数。
 - 自动内存管理: 注意看, 我们没有预先分配 outputs[j].buf 的内存。SDK 在内部帮我们 malloc 了一块足够大的 CPU 内存, 并把 NPU 的计算结果拷贝进去。
 - 自动反量化: 模型通常是 INT8 量化的, 但我们要算 Top5 需要浮点数。设置 want_float = 1 后, SDK 会在拷贝出来的同时, 利用模型里的 scale 和 zero_point 参数自动完成 INT8 -> FLOAT32 的数学转换。
- 资源释放 (rknn_outputs_release): 既然 SDK 帮我们 malloc 了内存, 我们就必须负责 free。如果不加这一行, 循环跑几次后程序就会因为内存泄漏而崩溃。这是新手最容易犯的错误。